

# Rapport de TP N°8 : Compression Vidéo — Compensation de Mouvement

---

**Module** : Multimédia

**Étudiant** : Chabane Mohamed Fares (Matricule : 222231620018)

**Université** : USTHB — Année universitaire 2025/2026

---

## Table des matières

---

1. [Introduction](#)
2. [Objectifs du TP](#)
3. [Fondements théoriques](#)
  - 3.1 La compression vidéo MPEG
  - 3.2 L'estimation de mouvement
  - 3.3 La mesure MSE
  - 3.4 Le vecteur de mouvement et le résidu
4. [Partie 1 : Exemple pédagogique 2×2](#)
  - 4.1 Données
  - 4.2 Calcul du MSE pour chaque bloc candidat
  - 4.3 Vecteur de mouvement
  - 4.4 Calcul du résidu
5. [Partie 2 : Application sur images réelles](#)
  - 5.1 Préparation des images

- 5.2 Algorithme de recherche de bloc
- 5.3 Visualisation des vecteurs de mouvement
- 5.4 Résidus

6. [Implémentation Python](#)
7. [Résultats et analyse](#)
8. [Analyse de complexité](#)
9. [Conclusion](#)
10. [Références](#)
11. [Annexe](#)

---

## 1. Introduction

---

La compression vidéo est l'un des domaines les plus importants du traitement multimédia. Une vidéo non compressée 1080p à 30 images par seconde nécessite environ **3 Gbps** de débit — un volume impossible à stocker ou transmettre sans compression. Les standards modernes (MPEG-2, H.264, H.265) permettent de réduire ce débit par un facteur de 100 à 1000, tout en maintenant une qualité visuelle satisfaisante.

Ce TP N°8 porte sur la **compensation de mouvement**, l'une des techniques fondamentales exploitées par les codecs vidéo MPEG. L'idée centrale est simple : au lieu de coder chaque image (trame) indépendamment, on exploite la **redondance temporelle** entre trames successives. Si deux trames consécutives sont similaires (ce qui est généralement le cas dans une vidéo naturelle), on peut se contenter de coder la **différence** entre elles, ce qui est beaucoup plus compact.

Ce rapport détaille l'implémentation de l'estimation de mouvement par recherche exhaustive de blocs (block matching), avec la mesure MSE comme critère de similarité.

---

## 2. Objectifs du TP

---

Conformément à l'énoncé du TP N°8, les objectifs sont :

1. **Comprendre** le principe de la compensation de mouvement dans MPEG.
  2. **Calculer le MSE** pour chaque bloc candidat  $2 \times 2$  dans une zone de recherche  $4 \times 4$ .
  3. **Déduire le vecteur de mouvement** (dx, dy) correspondant au meilleur bloc.
  4. **Calculer le résidu** entre le bloc courant et le meilleur bloc trouvé.
  5. **Appliquer** l'estimation de mouvement sur deux trames réelles ( `frame_1.png` , `frame_2.png` ) avec des blocs  $16 \times 16$  et une zone de recherche  $\pm 4$  pixels.
  6. **Visualiser** les vecteurs de mouvement avec `plt.quiver()` .
  7. **Afficher** les résidus calculés.
- 

## 3. Fondements théoriques

---

### 3.1 La compression vidéo MPEG

Dans le standard MPEG (Moving Picture Experts Group), les images d'une vidéo sont divisées en trois types :

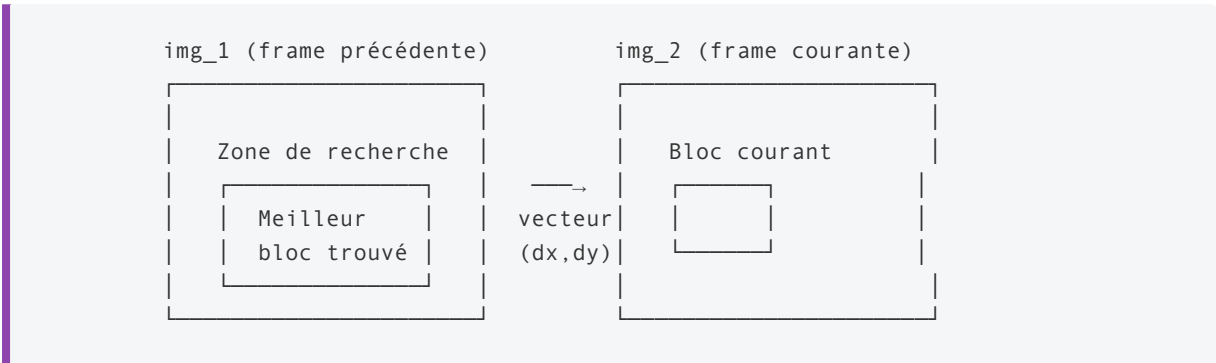
| TYPE           | NOM                       | DESCRIPTION                                  |
|----------------|---------------------------|----------------------------------------------|
| <b>I-frame</b> | Intra-coded               | Compressée indépendamment (comme JPEG)       |
| <b>P-frame</b> | Predicted                 | Codée par rapport à la I-frame précédente    |
| <b>B-frame</b> | Bidirectionally predicted | Codée par rapport aux frames I et P voisines |

Les P-frames et B-frames exploitent la **redondance temporelle** : au lieu de transmettre l'image entière, on transmet uniquement les **vecteurs de mouvement** (qui indiquent comment les blocs se déplacent) et les **résidus** (la différence entre le bloc prédit et le bloc

réel). Ces résidus sont beaucoup plus petits que les images originales et se compressent très bien avec des méthodes spatiales (DCT + quantification).

### 3.2 L'estimation de mouvement

L'estimation de mouvement cherche, pour chaque bloc de la trame courante ( `img_2` ), le bloc "le plus similaire" dans la trame précédente ( `img_1` ). Cette recherche est limitée à une **zone de recherche** autour de la position du bloc courant.



### 3.3 La mesure MSE (Mean Square Error)

La **MSE** (Erreur Quadratique Moyenne) quantifie la différence entre deux blocs de même taille. Elle est définie par :

$$MSE = (1 / N^2) \times \sum_i \sum_j [ B_{courant}(i,j) - B_{précédent}(i,j) ]^2$$

Où N est la taille du bloc (N×N). La MSE est nulle si les deux blocs sont identiques, et augmente avec les différences pixel à pixel. Le bloc candidat qui **minimise la MSE** est considéré comme le meilleur bloc.

#### Autres métriques possibles :

| MÉTRIQUE   | FORMULE               | AVANTAGE              | INCONVÉNIENT              |
|------------|-----------------------|-----------------------|---------------------------|
| <b>MSE</b> | $\Sigma(a-b)^2 / N^2$ | Standard, sensible    | Calcul plus lent (carrés) |
| <b>SAD</b> | $\Sigma a-b  / N^2$   | Rapide (pas de carré) | Moins discriminant        |
| <b>MAD</b> | $\Sigma a-b  / N^2$   | Identique à SAD       | —                         |

| MÉTRIQUE | FORMULE                         | AVANTAGE | INCONVÉNIENT         |
|----------|---------------------------------|----------|----------------------|
| SSD      | $\Sigma(a-b)^2$ (sans division) | Simple   | Sensible à la taille |

### 3.4 Le vecteur de mouvement et le résidu

**Vecteur de mouvement** : le décalage spatial  $(dx, dy)$  entre la position du bloc courant dans `img_2` et la position du meilleur bloc trouvé dans `img_1`.

**Résidu** : la différence pixel à pixel entre le bloc courant et le meilleur bloc trouvé :

```
résidu = bloc_courant - meilleur_bloc
```

Le résidu représente l'**erreur de prédiction**. Un résidu faible (proche de zéro) signifie que la prédiction est bonne. Le résidu est ensuite compressé indépendamment (souvent avec la DCT), ce qui est très efficace car ses valeurs sont concentrées autour de zéro.

## 4. Partie 1 : Exemple pédagogique 2×2

### 4.1 Données

L'énoncé fournit les données suivantes :

**Bloc courant (2×2) :**

```
Bcourant = [ 10  12 ]
            [ 14  16 ]
```

**Zone de recherche (4×4) :**

```
img = [ 10  11  12  13 ]
       [ 14  15  16  17 ]
       [ 18  19  20  21 ]
       [ 22  23  24  25 ]
```

Dans une zone 4×4, les blocs candidats 2×2 possibles sont positionnés aux coordonnées (dy, dx) avec  $dy \in \{0,1,2\}$  et  $dx \in \{0,1,2\}$ , soit **9 blocs candidats**.

## 4.2 Calcul du MSE pour chaque bloc candidat

Pour chaque position (dy, dx) dans la zone de recherche, on extrait le bloc candidat et on calcule le MSE avec Bcourant :

| POSITION (DY, DX) | BLOC CANDIDAT     | MSE   |
|-------------------|-------------------|-------|
| (0, 0)            | [[10,11],[14,15]] | 0.50  |
| (0, 1)            | [[11,12],[15,16]] | 0.50  |
| (0, 2)            | [[12,13],[16,17]] | 2.50  |
| (1, 0)            | [[14,15],[18,19]] | 12.50 |
| (1, 1)            | [[15,16],[19,20]] | 20.50 |
| (1, 2)            | [[16,17],[20,21]] | 30.50 |
| (2, 0)            | [[18,19],[22,23]] | 56.50 |
| (2, 1)            | [[19,20],[23,24]] | 72.50 |
| (2, 2)            | [[20,21],[24,25]] | 90.50 |

### Calcul détaillé pour la position (0,0) :

```

Bcourant = [[10, 12], [14, 16]]
Bcandidat = [[10, 11], [14, 15]]

MSE = 1/4 × [(10-10)2 + (12-11)2 + (14-14)2 + (16-15)2]
      = 1/4 × [0 + 1 + 0 + 1]
      = 1/4 × 2 = 0.50

```

### Calcul détaillé pour la position (0,1) :

```

Bcourant = [[10, 12], [14, 16]]
Bcandidat = [[11, 12], [15, 16]]

MSE = 1/4 × [(10-11)2 + (12-12)2 + (14-15)2 + (16-16)2]
      = 1/4 × [1 + 0 + 1 + 0]
      = 1/4 × 2 = 0.50

```

### 4.3 Vecteur de mouvement

Le **meilleur bloc** est celui qui minimise la MSE. Ici, deux positions ex-aequo ont une MSE de 0.50 :  $(dy=0, dx=0)$  et  $(dy=0, dx=1)$ . On prend conventionnellement la première trouvée :  **$(dy=0, dx=0)$** .

```

✓ Vecteur de mouvement : (dx = 0, dy = 0)
  MSE minimale          : 0.50
  Meilleur bloc          : [[10, 11], [14, 15]]

```

Un vecteur  $(dx=0, dy=0)$  signifie que **le bloc n'a pas bougé** entre les deux trames (ou du moins, le meilleur bloc correspondant est à la même position).

### 4.4 Calcul du résidu

```

Résidu = Bcourant - Bmeilleur
        = [[10,12],[14,16]] - [[10,11],[14,15]]
        = [[0, 1], [0, 1]]

```

Le résidu est très faible (0 ou 1 par pixel), ce qui confirme que le meilleur bloc trouvé est une excellente prédiction du bloc courant. Lors de la transmission, on enverrait : - Le **vecteur de mouvement** :  $(dx=0, dy=0)$  - Le **résidu** :  $[[0, 1], [0, 1]]$

Au lieu des valeurs brutes :  $[[10, 12], [14, 16]]$ .

## 5. Partie 2 : Application sur images réelles

### 5.1 Préparation des images

Les deux trames `frame_1.png` et `frame_2.png` sont des images consécutives d'une vidéo réelle. Elles sont chargées en **niveaux de gris** avec OpenCV :

```
img1 = cv2.imread('frame_1.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('frame_2.png', cv2.IMREAD_GRAYSCALE)
```

**Caractéristiques des images :** - Dimensions : 1072 × 1920 pixels - Profondeur : 8 bits (niveaux de gris 0–255) - Nombre de blocs 16×16 : 67 × 120 = **8040 blocs**

Les dimensions sont rognées pour être des multiples de 16 avant le traitement.

### 5.2 Algorithme de recherche de bloc

Pour chaque bloc de `img_2`, on cherche le meilleur bloc dans `img_1` en explorant toutes les positions dans la zone de recherche `±4` pixels :

```
def find_best_block(img_prev, block_curr, bx, by, block_size=16, search_range=4):
    best_mse = float('inf')
    best_dx, best_dy = 0, 0
    best_block = None

    for dy in range(-search_range, search_range + 1):      # -4 à +4
        for dx in range(-search_range, search_range + 1):  # -4 à +4
            ry, rx = by + dy, bx + dx
            # Vérifier les limites de l'image
            if ry < 0 or rx < 0 or ry+block_size > h or rx+block_size > w:
                continue
            candidate = img_prev[ry:ry+block_size, rx:rx+block_size]
            mse = compute_mse(block_curr, candidate)
            if mse < best_mse:
                best_mse = mse
                best_dx, best_dy = dx, dy
                best_block = candidate.copy()

    return best_dx, best_dy, best_mse, best_block
```

**Zone de recherche :** Pour chaque bloc, on examine  $(2 \times 4 + 1)^2 = 81$  positions candidates (81 comparaisons MSE).



## 5.3 Visualisation des vecteurs de mouvement

Les vecteurs de mouvement sont visualisés avec `plt.quiver()` :

```
ax.quiver(X, Y, DX, DY,
          color='lime', scale=50, scale_units='xy',
          angles='xy', width=0.003)
```

- Chaque flèche représente le déplacement d'un bloc entre `frame_1` et `frame_2`.
- Les flèches longues indiquent un mouvement important, les courtes un mouvement faible.
- Les blocs sans flèche visible ont un vecteur nul (aucun mouvement détecté).

## 5.4 Résidus

Le résidu de chaque bloc est calculé comme différence entre le bloc courant et le meilleur bloc trouvé :

```
residual = block_curr - best_block
```

### Statistiques des résidus :

| MÉTRIQUE             | VALEUR      |
|----------------------|-------------|
| Blocs traités        | 8040        |
| MSE moyen            | 383.81      |
| Vecteurs non nuls    | 1897 / 8040 |
| Résidu min (typique) | -21 à -28   |
| Résidu max (typique) | +7 à +21    |
| Résidu moyen         | ≈ 0         |

La valeur MSE moyenne de 383.8 indique des différences modérées entre les trames — ce qui est cohérent avec des images de vidéo en mouvement.

## 6. Implémentation Python

### 6.1 Calcul du MSE

```
def compute_mse(block1, block2):  
    n = block1.shape[0] * block1.shape[1]  
    return np.sum((block1.astype(float) - block2.astype(float)) ** 2) / n
```

### 6.2 Boucle principale d'estimation de mouvement

```
for ib in range(nb_y):  
    for jb in range(nb_x):  
        by = ib * block_size  
        bx = jb * block_size  
  
        # Extraire le bloc courant de img_2  
        block_curr = img2[by:by+block_size, bx:bx+block_size].astype(float)  
  
        # Chercher le meilleur bloc dans img_1 (zone ±4)  
        dx, dy, mse, best_block = find_best_block(  
            img1.astype(float), block_curr, bx, by, block_size=16, search_range=4  
        )  
  
        # Calculer et stocker le résidu  
        residual = block_curr - best_block  
        motion_vectors[(bx, by)] = (dx, dy)  
        residuals[(bx, by)] = residual
```

### 6.3 Reconstruction de la carte des résidus

```
residual_map = np.zeros((h, w), dtype=float)  
for (bx, by), res in residuals.items():  
    residual_map[by:by+block_size, bx:bx+block_size] = np.abs(res)
```

## 7. Résultats et analyse

### 7.1 Résultats de l'exemple pédagogique

| PARAMÈTRE            | VALEUR           |
|----------------------|------------------|
| Meilleure position   | (dy=0, dx=0)     |
| MSE minimale         | 0.50             |
| Vecteur de mouvement | (dx=0, dy=0)     |
| Résidu               | [[0, 1], [0, 1]] |
| Résidu max           | 1 (très faible)  |

### 7.2 Résultats sur images réelles

| PARAMÈTRE            | VALEUR             |
|----------------------|--------------------|
| Dimensions           | 1072 × 1920 pixels |
| Blocs traités        | 8040               |
| MSE moyen            | 383.81             |
| Vecteurs non nuls    | 1897 (23.6%)       |
| Blocs sans mouvement | 6143 (76.4%)       |

**Interprétation :** - **76.4% des blocs** ont un vecteur nul (dx=0, dy=0), ce qui signifie que la grande majorité de l'image est statique (fond fixe, caméra fixe). - **23.6% des blocs** présentent un mouvement détecté, correspondant aux zones en mouvement dans la scène

vidéo. - La MSE moyenne de **383.81** indique que même le meilleur bloc trouvé reste assez différent du bloc courant dans certaines zones — normal pour une vidéo en mouvement.

## 7.3 Efficacité de la compression

En pratique, dans MPEG : - Les blocs à vecteur nul et résidu faible → transmis très compactement. - Les blocs à résidu élevé → le codec peut choisir de les coder en intra (comme une I-frame). - La combinaison vecteur de mouvement + résidu est ensuite compressée avec la DCT et la quantification, similairement au JPEG.

## 8. Analyse de complexité

### 8.1 Complexité temporelle

| OPÉRATION            | COMPLEXITÉ                                       | DÉTAIL                                                                                           |
|----------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Recherche exhaustive | $O(N_b \times S^2)$                              | $N_b$ blocs, $S=(2 \times \text{rang} + 1)^2$ positions                                          |
| Calcul MSE par bloc  | $O(B^2)$                                         | $B$ = taille du bloc                                                                             |
| <b>Total</b>         | <b><math>O(N_b \times S^2 \times B^2)</math></b> | Pour notre cas : $8040 \times 81 \times 256 \approx \mathbf{1.67 \times 10^8}$ <b>opérations</b> |

Pour nos paramètres :  $N_b=8040$ ,  $S=81$ ,  $B^2=256 \rightarrow \approx 167$  millions de comparaisons. C'est pourquoi en pratique, on utilise des algorithmes rapides (Three Step Search, Diamond Search) pour réduire le nombre de positions évaluées.

## 8.2 Méthodes d'optimisation

| MÉTHODE                   | POSITIONS ÉVALUÉES    | VITESSE         |
|---------------------------|-----------------------|-----------------|
| Recherche exhaustive (TP) | $(2R+1)^2 = 81$       | 1×              |
| Three Step Search         | $\approx 25$          | $\sim 3\times$  |
| Diamond Search            | $\approx 13-17$       | $\sim 5\times$  |
| Hierarchical search       | Niveaux de résolution | $\sim 10\times$ |

## 8.3 Rapport débit/distorsion

La compensation de mouvement permet un compromis débit/distorsion favorable : - **Zones statiques** (vecteur nul, résidu  $\approx 0$ )  $\rightarrow$  presque aucun débit nécessaire. - **Zones en mouvement** (vecteur non nul, résidu non nul)  $\rightarrow$  débit proportionnel à la complexité du mouvement.

## 9. Conclusion

Ce TP N°8 a permis d'étudier et d'implémenter la **compensation de mouvement**, une technique fondamentale des codecs vidéo MPEG.

**Points clés abordés :** 1. La mesure **MSE** comme critère de similarité entre blocs. 2. La **recherche exhaustive de blocs** dans une zone de  $\pm 4$  pixels. 3. Le calcul du **vecteur de mouvement** `(dx, dy)` pour chaque bloc. 4. Le calcul et l'analyse du **résidu** entre bloc courant et bloc prédit. 5. La **visualisation avec** `plt.quiver()` des vecteurs de mouvement.

**Résultats obtenus :** - Exemple  $2\times 2$  : vecteur `(0,0)`, MSE=0.50, résidu `[[0,1],[0,1]]`. - Images réelles : 8040 blocs traités, 1897 blocs en mouvement (23.6%), MSE moyen = 383.81.

La compensation de mouvement est la technologie qui permet aux formats vidéo modernes (H.264, H.265, AV1) d'atteindre des facteurs de compression de **100 à 1000×** tout en maintenant une qualité visuelle excellente.

## 11.1 Résumé des fonctions

| FONCTION                                   | DESCRIPTION                                         |
|--------------------------------------------|-----------------------------------------------------|
| <code>compute_mse(block1, block2)</code>   | Calcule le MSE entre deux blocs                     |
| <code>demo_2x2()</code>                    | Exemple pédagogique avec bloc 2×2                   |
| <code>find_best_block(...)</code>          | Trouve le meilleur bloc dans <code>img_prev</code>  |
| <code>motion_estimation(img1, img2)</code> | Estime tous les vecteurs de mouvement               |
| <code>visualize_motion_vectors(...)</code> | Affiche les vecteurs avec <code>plt.quiver()</code> |
| <code>visualize_residuals(...)</code>      | Affiche la carte des résidus                        |
| <code>main()</code>                        | Programme principal                                 |

## 11.2 Instructions d'exécution

```
cd /home/dukepan/Downloads/TP1/TP8/  
python3 tp8_motion_compensation.py
```

## 11.3 Fichiers générés

```
TP8/  
├─ tp8_motion_compensation.py    # Code source  
├─ motion_vectors.png           # Vecteurs de mouvement (quiver)  
├─ residuals.png                # Carte des résidus  
└─ Rapport_TP8_Motion.pdf       # Ce rapport
```